

SQL Tokenizer

題目描述

請使用 flex 實作一個簡易的 SQL tokenizer，將 SQL 語句分解為不同類型的 token 並輸出相應的 token 類型和值。

輸入格式

輸入為一段 SQL 語句，可能包含以下元素：

- **KEYWORD**
 - 包含：SELECT, FROM, WHERE, AND, OR 等 5 個 SQL 保留字
 - 大小寫不敏感，即可接受 select, Select, sEleCt 等不同大小寫組合
- **IDENTIFIER**
 - 表格名、欄位名、變數名等
 - 只能包含大小寫英文字母、數字、底線（`_`）
 - 不能以數字開頭 *testcase #12*
 - 大小寫敏感
- **NUMBER**
 - 整數（包含負整數）
 - 輸入可以以零開頭（如：01, 007, -003）
 - 數值範圍介於 -2^{31} 與 $2^{31} - 1$ 之間
- **STRING**
 - 以單引號包圍的字串
 - 支援單引號轉義（兩個連續單引號 `''` 表示一個單引號字元）
 - 可包含除了換行符（`\n`）以外的所有字元，同時題目輸入的單引號中亦不會包含換行符
- **OPERATOR**
 - 包含：*, =, >, <, >=, <=, !=
- **PUNCTUATION**
 - 包含：,,;

另外，SQL 語句中可含有註解，在處理時，請忽略所有註解文字與註解符號本身。註解有兩種形式：

- 單行註解
 - 以連續的兩個連字符 `--` 開始單行註解
- 多行註解
 - 於註解符號 `/*` 與 `*/` 中的內容為多行註解
 - 多行註解可為巢狀結構 *testcase #9*
 - 題目輸入的註解符號 `/*` 與 `*/` 必定順序成對出現

本題保證輸入符合以下限制：

- 總 token 數小於 50
- 每個 token 值的字串長度小於 100
- 若 token 為 **NUMBER**，其範圍介於 -2^{31} 與 $2^{31} - 1$ 之間
即測資不會輸入數學上定義為整數，但超出此範圍的數字

輸出格式

正常情況：對於每個識別出的 token，輸出兩行：

1. 第一行：token 的類型
2. 第二行：token 的值，有些 token 需要先進行處理再輸出
 - **OPERATOR**, **PUNCTUATION**, **IDENTIFIER**：直接輸出原始字串
 - **KEYWORD**：輸出時需轉為大寫 *testcase #4*
 - **NUMBER**：刪除多餘的零與不必要的負號後輸出 *testcase #10*
 - 01 -> 1
 - -003 -> -3
 - -0 -> 0

- **STRING** : 去除外層單引號 *testcase #5*，並將轉義的單引號還原後輸出 *testcase #6*

錯誤情況：若輸入中包含任何無效字元，則僅輸出一行 **INVALID** *testcase #11*

注意：一旦遇到無效字元，即使前面有正確的 token，也不輸出任何內容，僅輸出 **INVALID**。

Note

本題只需要實作詞法標記化（tokenization），**無須考慮 SQL 語法本身的合法性**。例如，即使輸入的 SQL 語句在語法上不正確（如語序錯誤），只要每個個別的 token 都是有效的字元組合，就應該正常輸出對應的 token 類型和值。語法分析（syntax analysis）不在本題的考試範圍內。

你可能會想使用一些 C 的字串處理函數，在題目最後有截取 `string.h` 與 `stdio.h` 文件的一小部分，提供給需要的同學參考。

公開測試資料

#	Input	Output
1	<pre>SELECT * FROM users;</pre>	<pre>KEYWORD SELECT OPERATOR * KEYWORD FROM IDENTIFIER users PUNCTUATION ;</pre>
2	<pre>SELECT name , age FROM employees WHERE id = 10 ;</pre> （輸入含有空格、<code>\t</code>、<code>\n</code>）	<pre>KEYWORD SELECT IDENTIFIER name PUNCTUATION , IDENTIFIER age KEYWORD FROM IDENTIFIER employees KEYWORD WHERE IDENTIFIER id OPERATOR = NUMBER 10 PUNCTUATION ;</pre>

3	<pre>SELECT select_id, from_user FROM _select_table;</pre>	<pre>KEYWORD SELECT IDENTIFIER select_id PUNCTUATION , IDENTIFIER from_user KEYWORD FROM IDENTIFIER _select_table PUNCTUATION ;</pre>
4	<pre>select NaMe from USERS where iD = 5;</pre>	<pre>KEYWORD SELECT IDENTIFIER NaMe KEYWORD FROM IDENTIFIER USERS KEYWORD WHERE IDENTIFIER iD OPERATOR = NUMBER 5 PUNCTUATION ;</pre>
5	<pre>SELECT 'Hello, world!' FROM messages;</pre>	<pre>KEYWORD SELECT STRING Hello, world! KEYWORD FROM IDENTIFIER messages PUNCTUATION ;</pre>
6	<pre>SELECT name FROM users WHERE name = 'O'Malley';</pre>	<pre>KEYWORD SELECT IDENTIFIER name KEYWORD FROM IDENTIFIER users KEYWORD WHERE IDENTIFIER name OPERATOR = STRING O'Malley PUNCTUATION ;</pre>

7	<pre>SELECT 'string with -- and /* symbols' FROM test;</pre>	KEYWORD SELECT STRING string with -- and /* symbols KEYWORD FROM IDENTIFIER test PUNCTUATION ;
8	<pre>SELECT * FROM data WHERE value > 10 AND value <= 20 OR value != 30;</pre>	KEYWORD SELECT OPERATOR * KEYWORD FROM IDENTIFIER data KEYWORD WHERE IDENTIFIER value OPERATOR > NUMBER 10 KEYWORD AND IDENTIFIER value OPERATOR <= NUMBER 20 KEYWORD OR IDENTIFIER value OPERATOR != NUMBER 30 PUNCTUATION ;
9	<pre>-- This is a single line comment SELECT /* This is a comment spanning multiple lines */ id FROM accounts; -- This is a comment at the end of a line /* Becareful /* Nested comment */ ! */</pre>	KEYWORD SELECT IDENTIFIER id KEYWORD FROM IDENTIFIER accounts PUNCTUATION ;

10	SELECT id, -5, 07, -003, 0 FROM numbers WHERE value > -10;	KEYWORD SELECT IDENTIFIER id PUNCTUATION , NUMBER -5 PUNCTUATION , NUMBER 7 PUNCTUATION , NUMBER -3 PUNCTUATION , NUMBER 0 KEYWORD FROM IDENTIFIER numbers KEYWORD WHERE IDENTIFIER value OPERATOR > NUMBER -10 PUNCTUATION ;
11	SELECT name FROM users WHERE email @ 'test.com';	INVALID
12	SELECT 123user FROM users;	KEYWORD SELECT NUMBER 123 IDENTIFIER user KEYWORD FROM IDENTIFIER users PUNCTUATION ;

C Reference

string.h

String manipulation

- `strcpy` : copies one string to another
 - Definition

```
char* strcpy( char* restrict dest, const char* restrict src );
```

- Parameters
 - `dest` - pointer to the character array to write to
 - `src` - pointer to the null-terminated byte string to copy from
- Return value
 - returns a copy of `dest`
- `strncpy` : copies a certain amount of characters from one string to another
 - Definition

```
char *strncpy( char *restrict dest, const char *restrict src, size_t count );
```

- Parameters
 - `dest` - pointer to the character array to copy to
 - `src` - pointer to the character array to copy from
 - `count` - maximum number of characters to copy
- Return value
 - returns a copy of `dest`
- `strcat` : concatenates two strings
 - Definition

```
char *strcat( char *restrict dest, const char *restrict src );
```

- Parameters
 - `dest` - pointer to the null-terminated byte string to append to
 - `src` - pointer to the null-terminated byte string to copy from
- Return value
 - returns a copy of `dest`
- `strncat` : concatenates a certain amount of characters of two strings
 - Definition

```
char *strncat( char *restrict dest, const char *restrict src, size_t count );
```

- Parameters
 - `dest` - pointer to the null-terminated byte string to append to
 - `src` - pointer to the character array to copy from
 - `count` - maximum number of characters to copy
- Return value
 - returns a copy of `dest`

String examination

- `strlen` : returns the length of a given string
 - Definition

```
size_t strlen( const char* str );
```

- Parameters
 - `str` - pointer to the null-terminated byte string to examine
- Return value
 - returns the length of the null-terminated byte string `str`

Character array manipulation

- `memset` : fills a buffer with a character
 - Definition

```
void *memset( void *dest, int ch, size_t count );
```

- Parameters
 - `dest` - pointer to the object to fill
 - `ch` - fill byte
 - `count` - number of bytes to fill
- Return value
 - returns a copy of `dest`
- `memcpy` : copies one buffer to another
 - Definition

```
void* memcpy( void *restrict dest, const void *restrict src, size_t count );
```

- Parameters
 - `dest` - pointer to the object to copy to
 - `destsz` - max number of bytes to modify in the destination (typically the size of the destination object)
 - `src` - pointer to the object to copy from
- Return value
 - returns a copy of `dest`

stdio.h

Formatted input/output

- `printf`, `sprintf` : prints formatted output to stdout or a buffer

- Definition

```
int printf( const char* restrict format, ... );  
int sprintf( char* restrict buffer, const char* restrict format, ... );
```

- Parameters
 - `buffer` - pointer to a character string to write to
 - `format` - pointer to a null-terminated byte string specifying how to interpret the data
 - `...` - arguments specifying data to print. If any argument after default argument promotions is not the type expected by the corresponding conversion specification (the expected type is the promoted type or a compatible type of the promoted type), or if there are fewer arguments than required by `format`, the behavior is undefined. If there are more arguments than required by `format`, the extraneous arguments are evaluated and ignored.
- Return value
 - a. for `printf` : returns number of characters transmitted to the output stream or negative value if an output error or an encoding error (for string and character conversion specifiers) occurred.
 - b. for `sprintf` : number of characters written to `buffer` (not counting the terminating null character), or a negative value if an encoding error (for string and character conversion specifiers) occurred.

- `scanf`, `sscanf` : reads formatted input from stdin or a buffer

- Definition

```
int scanf( const char *restrict format, ... );  
int sscanf( const char *restrict buffer, const char *restrict format, ... );
```

- Parameters
 - `buffer` - pointer to a null-terminated character string to read from
 - `format` - pointer to a null-terminated character string specifying how to read the input
 - `...` - receiving arguments.
- Return value

returns number of receiving arguments successfully assigned (which may be zero in case a matching failure occurred before the first receiving argument was assigned), or `E0F` if input failure occurs before the first receiving argument was assigned.

stdlib.h

- `atoi` : converts a string to an integer with base 10

- Definition

```
int atoi( const char *str );
```

- Parameters
 - `str` - pointer to a null-terminated byte string representing an integer number
- Return value
 - returns the converted integer value

Note: To convert integer to string, you may use `sprintf` from `stdio.h`.